

Final Assignment: System Integration – Building an IoT Control API

Course: Systemintegration

Due Date: 2025-08-22

1. Introduction

This assignment is the culmination of your work in the System Integration course. Its purpose is for you to apply the knowledge you've gained to design, develop, document, and evaluate a complete web service in a realistic IoT context. You will demonstrate your skills in creating effective APIs, working with service-oriented principles, and handling structured data, as well as your competence in independently managing a software project from specification to reflection.

This project directly assesses your proficiency in the course's learning objectives, particularly Färdigheter (Skills) 6–11 and Kompetenser (Competencies) 12–13.

2. The Core Task

Your task is to create a RESTful API using Python, FastAPI, and Pydantic. This API will serve as the backend for a simulated IoT system. You will need to define the resources, create Pydantic models for data validation, implement a full set of CRUD (Create, Read, Update, Delete) endpoints, and provide clear documentation.

You will choose one of the following scenarios to build your API for.

3. Scenario Options

Option A: Smart Home Hub API (Primary Option)

Context: You are building the central API for a smart home hub. This API allows a mobile app or a web dashboard to register, monitor, and control various smart devices within a home.

Resource to Model: A Device.

- Pydantic Model Fields:
 - `id` (integer, server-assigned)
 - `name` (string, e.g., "Living Room Lamp")
 - `type` (string, must be one of: "light", "thermostat", "smart_plug")

- `location` (string, e.g., "Kitchen")
- `is_on` (boolean, representing the on/off state)
- `value` (Optional float, e.g., brightness for a light (0–100), temperature for a thermostat)

Option B: Environmental Sensor Network API (Alternative)

Context: You are creating the API for a network of environmental sensors deployed across a city. This API collects data and manages the sensor nodes.

Resource to Model: A `SensorNode`.

- Pydantic Model Fields:
 - `id` (integer, server-assigned)
 - `mac_address` (string, a unique hardware identifier)
 - `location` (string, e.g., "Gamla Stan")
 - `last_seen` (string, representing a datetime)
 - `status` (string, must be one of: "active", "inactive", "maintenance")
 - `latest_temp` (Optional float, the last reported temperature)

Option C: Warehouse Asset Tracker API (Alternative)

Context: You are building the API for a smart warehouse system that uses IoT tags to track high-value assets.

Resource to Model: An `Asset`.

- Pydantic Model Fields:
 - `id` (integer, server-assigned)
 - `tag_id` (string, the unique ID of the IoT tag attached to the asset)
 - `name` (string, e.g., "Industrial Power Drill")
 - `last_location` (string, e.g., "Aisle 4, Shelf B")
 - `last_update_timestamp` (string, representing a datetime)
 - `is_moving` (boolean)

4. Technical Requirements

Your API must meet the following requirements, regardless of the scenario you choose.

4.1. API Functionality (CRUD Endpoints)

You must implement a complete set of RESTful endpoints for your chosen resource (e.g., `/devices/`).

- `POST /<resource>` : Creates a new resource.
- `GET /<resource>` : Retrieves a list of all resources.
- `GET /<resource>/{id}` : Retrieves a single resource by its unique ID.
- `PUT /<resource>/{id}` : Fully updates an existing resource.
- `DELETE /<resource>/{id}` : Deletes a resource.

4.2. Data Modeling and Validation

- You must use Pydantic `BaseModel`s to define the schema for your resource.
- Create separate models for creation (e.g., `DeviceCreate`, without the `id`), full representation (e.g., `Device`, with the `id`), and updates (e.g., `DeviceUpdate`, with optional fields).
- Your API must use these models for request body validation and as `response_model`s to ensure consistent output.

4.3. Error Handling

- Your API must handle common error cases gracefully. Specifically, if a client requests, updates, or deletes a resource with an ID that does not exist, your API must return a proper `404 Not Found` error with a descriptive message, using FastAPI's `HTTPException`.

4.4. Code Quality and Documentation

- Your Python code must be clean, well-structured, and commented where necessary to explain complex logic.
- Use meaningful variable and function names.
- Your project should have a `README.md` file explaining what the API does and how to run it.

4.5. (Bonus for VG Grade) Partial Updates

- Implement a `PATCH /<resource>/{id}` endpoint that allows for partial updates (i.e., updating only the fields provided in the request body, leaving others unchanged). This will require a separate Pydantic model with all fields marked as `Optional`.

5. Submission Requirements

You must submit a single `.zip` file containing:

1. Your complete project folder, including:
 - Your main Python file (e.g., `main.py`).
 - A `requirements.txt` file listing the necessary packages (`fastapi` , `uvicorn` , etc.).
 - The `README.md` file.
2. A written report in PDF format (see details below).

6. The Written Report

Your report is a critical part of the assignment where you demonstrate your understanding and reflect on your work. It should be approximately 1–2 pages and include the following sections:

1. Introduction:
 - Briefly state which scenario you chose and why.
 - Provide a high-level overview of the API you built.
2. Design and Planning:
 - Explain the design choices you made for your API. Why did you structure your Pydantic models the way you did?
 - Describe your API's URL structure (e.g., `/devices` , `/devices/{id}`).
3. API Endpoint Documentation:
 - Provide a table documenting each of your endpoints. For each endpoint, list:
 - The HTTP Method (e.g., `POST`)
 - The URL Path (e.g., `/devices/`)
 - A brief description of what it does.
 - The expected request body schema (if any).
 - The successful response code and schema.
4. Challenges and Solutions:
 - Describe at least one technical challenge you faced during the project (e.g., a logical error, a validation issue, a problem with your data structure).
 - Explain how you identified and solved the problem. What resources did you use to find the solution (e.g., FastAPI documentation, Stack Overflow, lecture notes)?
5. Evaluation and Reflection:
 - Critically evaluate your own solution. What are its limitations?

- Propose at least two concrete improvements for a "Version 2.0" of your API. Explain why these improvements would be valuable. (Examples: adding authentication, connecting to a real database, implementing more complex business logic, adding filtering/sorting to your `GET` list endpoint).

7. Assessment Criteria

This assignment will be graded as Icke Godkänt (IG), Godkänt (G), or Väl Godkänt (VG).

- To achieve Godkänt (G):
 - All technical requirements in section 4 (excluding the bonus) must be met.
 - The API must be fully functional and perform all five CRUD operations correctly.
 - The submitted code must be well-structured and run without errors.
 - The written report must be complete and address all sections with clear, coherent explanations.
- To achieve Väl Godkänt (VG):
 - All requirements for a G grade must be met.
 - In addition, you must demonstrate a deeper level of understanding and independence. This can be shown through a combination of the following:
 - Successfully implementing the bonus `PATCH` endpoint.
 - Exceptionally high code quality, with clear documentation and structure.
 - A particularly insightful and well-argued "Evaluation and Reflection" section in your report that shows a strong grasp of architectural trade-offs and future development paths.
 - Implementing an additional valuable feature not explicitly required (e.g., a custom validator, query parameters for filtering the `GET` list).